

A Polynomial-Time Algorithm for the Maximum Clique Problem

Zohreh O. Akbari

Department of Mathematics and Computer Science
Friedrich Schiller University
Jena, Germany
zohreh.akbari@uni-jena.de

Abstract—After more than six decades of its introduction, the maximum clique problem, which is one of the most applicable problems in the graph theory, has still no polynomial-time solution. This paper presents a polynomial-time algorithm for this problem, which detects the maximum clique of a given graph through a recursive approach. This polynomial solution to the clique problem, as an NP-complete problem, causes every problem in NP to have a polynomial solution, which leads to the equality of P and NP complexity classes.

Keywords—Computational Complexity Theory; Maximum Clique Problem; NP-Complete Problems; P versus NP Problem; Polynomial-Time Algorithm; Recursive Optimization

I. INTRODUCTION

The maximum clique problem is a classical problem in combinatorial optimization with important applications in different domains [1, 2]. The "clique" terminology was first used in the field of computer science by [3] in 1949. After several discussions of the problem related to this concept in [4, 5, 6, 7] the first algorithm for solving the clique problem was according to [1, 2] introduced by [8] in 1957.

Since the work of [8], many others have devised algorithms for various versions of the clique problem [1, 2]. In the 1970s, beginning with the work of [9] and [10], researchers began finding mathematical justification for the perceived difficulty of the clique problem in the theory of NP-completeness. They also began studying the algorithms concerning this problem from the point of view of worst-case analysis [11].

NP-Complete problems are typically solved using techniques such as: Approximation, Randomization, Parameterization, Restriction, and Heuristic algorithms. These algorithms give rise to substantially faster approaches in solving NP-Complete problems; however they do not resolve the problem [12].

The computational complexity of approximating the maximum clique problem has been studied for a long time; however, little more was known until the early 1990s, when a breakthrough series of papers beginning with [13] began to prove the hardness of approximation results for the maximum clique problem [14, 15, 16, 17]. After many improvements to these results it is now known that, although some NP-complete problems such as the bin packing problem, can be

approximated within any factor greater than 1 [18] others such as the maximum clique problem are impossible to approximate within any constant, or even polynomial factor, unless $P = NP$. There can be no polynomial time algorithm that approximates the maximum clique to within a factor better than $O(n^{1-\epsilon})$, for any $\epsilon > 0$ [19]. This shows the necessity of a polynomial algorithm for this problem.

An important consequence of the Cook-Levin theorem [9, 20] claims, that if any NP-complete problem can be solved in polynomial time, then every problem in NP has a polynomial-time solution [21]. Thus besides many important applications in different domains, a polynomial-time solution to the maximum clique problem, as an NP-complete problem causes every problem in NP to have a polynomial solution, which leads to the equality of P and NP complexity classes.

The P versus NP problem, introduced in 1971 by Stephen Cook [9], is considered by many to be the most important open problem of our time and its importance grows with the rise of powerful computers [22]. A comprehensive list of claimed solutions to P versus NP problem can be found in [23].

This paper presents a polynomial-time algorithm for the maximum clique problem, which detects the maximum clique of a given graph through a recursive approach. The formal definition of the maximum clique problem is given in section II. Section III presents a polynomial-time algorithm followed by the proof of correctness in section IV. The time complexity of the algorithm is calculated in section V. Section VI concludes the results of the paper.

II. THE MAXIMUM CLIQUE PROBLEM

A clique in an undirected graph G is a complete subgraph of G , and a maximum clique is a clique that includes the largest possible number of vertices. In the maximum clique problem, the input is an undirected graph, and the output is a maximum clique in the graph.

A. The Formal Definition of the Problem

Through this paper, $G = (V, E)$ is an arbitrary undirected graph, where $V = \{1, 2, \dots, n\}$ is the vertex set of G and $E \subseteq V \times V$ is the edge set of G .

A graph $G = (V, E)$ is complete if all its vertices are pairwise adjacent, i.e. $\forall i, j \in V$ with $i \neq j$, we have $(i, j) \in E$. A

clique C is a subset of V such that $G(C)$ is complete. The clique number of G , denoted by $\omega(G)$ is the size of the maximum clique. The maximum clique problem asks for cliques of maximum cardinality (the cardinality of set S , i.e., the number of its elements which will be denoted by $|S|$) [1]:

$$\omega(G) = \max \{|S|: S \text{ is a clique in } G\}$$

III. A POLYNOMIAL-TIME ALGORITHM FOR THE MAXIMUM CLIQUE PROBLEM

The key idea behind our algorithm is to prune the graph in order to reach the maximum clique. Following this idea, the algorithm keeps removing the vertices of lowest degree from the graph on each step. Before removing a vertex it is necessary to calculate the maximum clique containing this vertex. This is gained through a recursive approach.

A. The Pseudo-Code of the Algorithm

The following is the coding of the algorithm in pseudo-code:

```

MaxClique(G)
{
1: if (G is a complete graph)
2:   for each vertex of G: v
3:     if ( $|V|-1 > \max C[v]$ )
4:        $\max C[v] := |V|$ ;
5:       make  $\max CP[v]$  point to a linked list containing  $V$ ;
6:   else
7:     find the vertex of lowest degree:  $\alpha$ 
8:     find the largest subgraph of G in which  $\alpha$  exists:  $G' (V', E')$ 
9:     MaxClique( $G'$ );
10:  if ( $V - \alpha \neq \emptyset$ )
11:    MaxClique( $G - \alpha$ );
}
```

At the beginning of each execution, the algorithm checks if the input graph happens to be a clique itself. If the input graph is a complete graph, or in other words, if the degree of every vertices of G is equal to $|V|-1$ the maximum clique of the input graph is found. In this case, for each vertex of G , the algorithm checks if there is no larger clique found for this vertex and then it saves the number of vertices and the subgraph as well for such vertices (An array named $\max C$ keeps the size of maximum clique found for each vertex and an array of pointers called $\max CP$, would point to the linked list containing the vertices of maximum clique for each vertex).

But in case that G is not a complete graph, the algorithm prunes it until reaching the maximum clique of it. It continues with finding the maximum cliques of two different subgraphs. First it finds the vertex of lowest degree. The next step is to find the largest subgraph in which the mentioned vertex exists. This can be easily done by considering all the vertices, which are connected to this vertex and all the relevant edges. This

would form the first subgraph which leads to the maximum clique containing α . The second subgraph would be $G - \alpha$. The detection of the maximum clique for each vertex leads to finding the maximum clique of the whole Graph.

B. An example of the algorithm's approach

The following example illustrates the algorithm's approach to solve a problem. As the input an arbitrary graph is considered as follows:

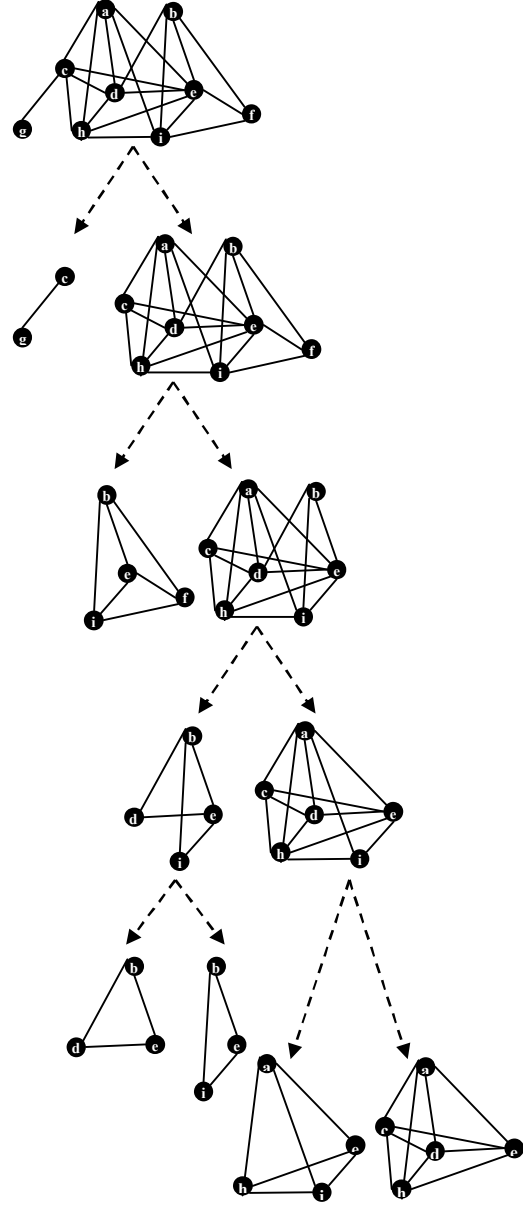


Fig. 1. The algorithm's approach to find the maximum clique of an arbitrary graph

At the end of the algorithm's execution the $\max C$ array contains the size of maximum clique found for each vertex and

the maxCP array would point to the linked list containing the vertices of maximum clique for each vertex.

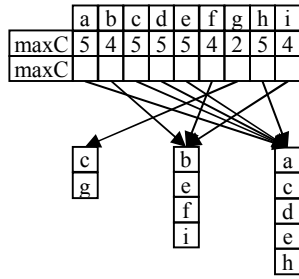


Fig. 2. The algorithm's output for the arbitrary graph of Fig. 1.

IV. THE PROOF OF CORRECTNESS OF THE ALGORITHM

The correctness of the algorithm may be shown using mathematical induction method. First we need to formulate the proposition for the algorithm correctness. In this case, we let $P(n)$ stand for the proposition that the algorithm finds and returns the maximum clique for graphs of size n . Accordingly, it is to show that $\forall n P(n)$ is true.

BASIS: When there is only one vertex in the graph, i.e., $|V|=1$, then since this graph is a complete graph of size 1, the maximum clique can be found at the first execution of the algorithm and the algorithm stops searching. It is thus clear that $P(1)$ is true.

INDUCTIVE STEP: Assume that the algorithm finds and returns the maximum clique when there are at most k vertices in a graph.

Now consider the case in which there are $(k + 1)$ vertices in a graph. If the graph is a complete graph itself, the maximum clique can be found just on the first execution of the algorithm.

In case of an incomplete graph, any vertex of the graph, which is first detected as the vertex of lowest degree, would cause as has been mentioned, a subgraph of size k at the most. Removing this vertex from the graph, we would have the second subgraph which is of size k . From the inductive hypothesis; we know that the algorithm is able to return the maximum clique in any graph of size 1 to k through recursive calls. Thus the algorithm is clearly able to return the maximum clique of both subgraphs and hence the maximum clique of the graph of size $k+1$ as well, since the answer should appear in one of these subgraphs. By applying the first principle of mathematical induction, we can conclude that $\forall n P(n)$ is true, i.e., the algorithm is correct.

V. THE TIME COMPLEXITY OF THE ALGORITHM

The maximum clique problem is an optimization problem. In such problems we are trying to find the optimum solution from all feasible solutions. A recursive divide and conquer approach is often successful for dealing with such problems. Being able to divide an optimization problem into subproblems in the way in which no feasible solutions are disregarded, i.e. one of the subproblems leads to the optimum solution; the optimization problem is to be solved recursively.

A recursive optimization algorithm for an NP optimization problem should be able to divide the problem into subproblems in polynomial time, and the number of its recursive calls should also be polynomial in the input size. According to the definition of the NP-problems, it is clear that the verification of possible answers could also be done in polynomial time.

Since maxClique(G) algorithm is a recursive algorithm, the time complexity of this algorithm is to be calculated as follows:

The time complexity of each execution of maxClique(G) $\times$ the number of recursive calls

A. The time complexity of each execution of maxClique(G)

Supposing that $T(L_i)$ indicates the time complexity of the execution of the i -th line of the algorithm, the time complexity of each execution of maxClique(G) is calculated as follows:

$$\sum_{i=1}^{11} T(L_i)$$

The first line of the algorithm checks if the input graph is a complete graph or not. To check if G is a complete graph, it would be enough to check whether the degree of every vertices of G is equal to $|V|-1$. Hence it is clear that $T(L_1) = O(n^2)$.

The loop between line 2 and line 5 takes $O(n^2)$, since $T(L_3)$, $T(L_4)$ and $T(L_5)$ are done respectively in $O(1)$, $O(1)$ and $O(n)$, and these lines are repeated $O(n)$ times in the loop.

Having the degree of each vertex (calculated on the first line), the vertex of lowest degree can be found in $O(n)$ and thus $T(L_7) = O(n)$.

In order to find the largest subgraph of G containing α , all we need to do is to consider the index of all the vertices which are connected to α in a structure such as a linked list. The edges can be easily checked using the adjacency matrix of graph G . Therefore it is clear that $T(L_8) = O(n)$.

Therefore the time complexity of each execution of maxClique(G) is as follows:

$$\sum_{i=1}^{11} T(L_i) = O(n^2)$$

Since the input of the algorithm is actually a matrix which carries a graph, the input size is n^2 . In other words, the time complexity of each execution of the algorithm is linear in the input size.

B. The number of recursive calls

The algorithm calls itself recursively at line 9 and 11, but the number of such calls is polynomial in the input size. The correctness of this claim may be shown using mathematical induction method as well. Let $P(n)$ stand for the proposition that the number of recursive calls to find and return the maximum clique of a graph with n vertices is polynomial in the input size. Accordingly, we have to show that $\forall n P(n)$ is true.

BASIS: When there is only one vertex in the graph, i.e., $|V|=1$, then since this vertex forms a complete graph, the maximum clique is founded on the first execution of the

algorithm and it stops searching. Therefore the algorithm is executed just once and thus $P(1)$ is true.

INDUCTIVE STEP: Assume that the number of recursive calls for the algorithm to find and return the maximum clique is polynomial in the input size of k^2 , when there are k vertices in a graph.

Now consider the case in which there are $(k + 1)$ vertices in a graph. If the graph is a complete graph itself, the maximum clique can be found just on the first execution of the algorithm.

In case of an incomplete graph, any vertex detected as the vertex of lowest degree, would cause a subgraph of maximum size k , since the given graph is not a complete graph. According to the inductive hypothesis, it is clear that the algorithm returns the maximum clique in number of recursive calls which is polynomial in the input size of k^2 . Removing the vertex of lowest degree, the new graph is also a graph of maximum size k , which causes number of recursive calls which is polynomial as well.

Thus the algorithm returns the maximum clique of a graph with $(k+1)$ vertices in number of recursive calls which is polynomial as well. By applying the first principle of mathematical induction, we can conclude that $\forall n P(n)$ is true, i.e., the algorithm calls itself recursively in polynomial number of calls in order to detect the maximum clique of a graph with n vertices.

Since the time complexity of each execution of the algorithm is linear in the input size and the algorithm calls itself recursively in polynomial number of calls in the input size the time complexity of the algorithm is also polynomial.

The implementation of the algorithm also shows that the number of recursive calls does not exceed $O(n^4)$, which is actually the second power of the input size. Thus it is seen that the time complexity of the algorithm does not exceed the third power of the input size. Following diagrams illustrate the number of recursive calls for random graphs of different sizes.

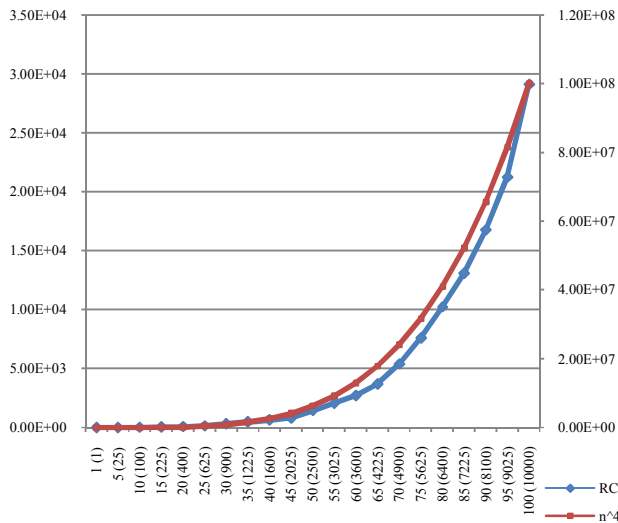


Fig. 3. The number of recursive calls for random graphs of different sizes between 1 to 100.

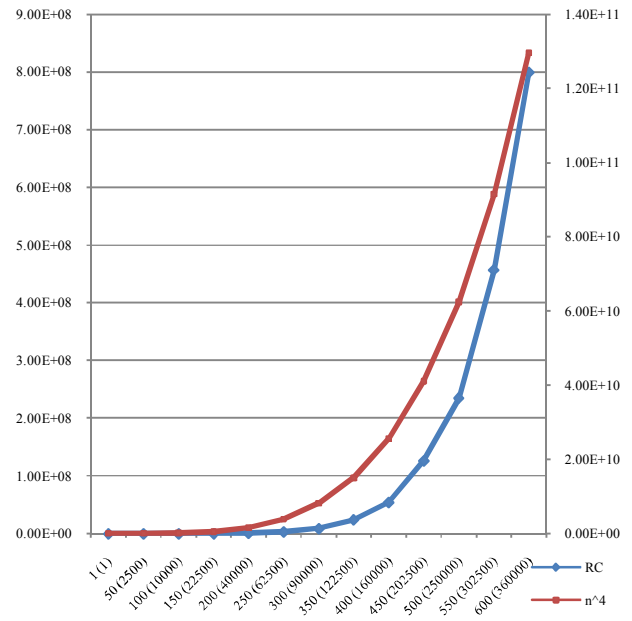


Fig. 4. The number of recursive calls for random graphs of different sizes between 100 to 600.

VI. CONCLUSION

The polynomial-time algorithm presented in this paper detects the maximum clique of a given graph in power three of the input size through a recursive approach. Thus is the clique problem the first NP-complete problem, for which, a polynomial-time algorithm is developed. An important consequence of the Cook-Levin theorem [9, 20] claims that if any NP-complete problem can be solved in polynomial time, then every problem in NP has a polynomial-time solution, that is, $P=NP$ [21].

Though this paper proves that the clique problem is to be solved in polynomial time in the input size, it fails to present a precise time order for the algorithm presented. As a future work this aim is to be considered. The general approach of the recursive optimization algorithm which is presented in this paper is also to be used to solve other NP optimization problems.

REFERENCES

- [1] I.M. Bomze, M. Budinich, P.M. Pardalos, M. Pelillo, "The maximum clique problem", Handbook of Combinatorial Optimization, 4, Kluwer Academic Publishers, pp. 1–74, 1999.
- [2] G. Gutin, "5.3 Independent sets and cliques", in J.L. Gross, J. Yellen, Handbook of graph theory, Discrete Mathematics & Its Applications, CRC Press, pp. 389–402, 2004.
- [3] R.D. Luce, A.D. Perry, "A method of matrix analysis of group structure", Psychometrika, 14(2): pp. 95–116, 1949.
- [4] L. Festinger, "The Analysis of Sociograms Using Matrix Algebra", Human Relations, 2, pp. 153–158, 1949.
- [5] E. Forsyth, L. Katz, "A Matrix Approach to the Analysis of Sociometric Data: Preliminary Report", Sociometry, 9, pp. 340–347, 1946.
- [6] R.D. Luce, "Connectivity and Generalized Cliques in Sociometric GroupStructure", Psychometrika, 15, pp. 169–190, 1950.

- [7] R.S. Weiss, E. Jacobson, "A Method for the Analysis of the Structure of Complex Organizations", *American Sociological Review*, 20, pp. 661-668, 1955.
- [8] F. Harary, I.C. Ross, "A procedure for clique detection using the group matrix", *Sociometry (American Sociological Association)* 20(3): 205-215, 1957.
- [9] S.A. Cook, "The complexity of theorem-proving procedures", *Proc. 3rd ACM Symposium on Theory of Computing*, pp. 151-158, 1971.
- [10] R.M. Karp, "Reducibility among combinatorial problems", in R.E. Miller, J.W. Thatcher, *Complexity of Computer Computations*, New York: Plenum, pp. 85-103, 1972.
- [11] R.E. Tarjan, A.E. Trojanowski, "Finding a maximum independent set", *SIAM Journal on Computing* 6 (3): 537-546, 1977.
- [12] M. Garey, D. Johnson, *Computers and Intractability: A Guide to the Theory of NP-Completeness*, New York: W. H. Freeman and Co, 1979.
- [13] U. Feige, S. Goldwasser, L. Lovász, S. Safra, M. Szegedy, "Approximating clique is almost NP-complete", *Proc. 32nd IEEE Symp. on Foundations of Computer Science*, pp. 2-12, 1991.
- [14] G. Kolata, "In a Frenzy, Math Enters Age of Electronic Mail", *New York Times*, June 26, 1990.
- [15] U. Feige, S. Goldwasser, L. Lovász, S. Safra, M. Szegedy, "Approximating clique is almost NP-complete", *Proc. 32nd IEEE Symp. on Foundations of Computer Science*, pp. 2-12, 1991.
- [16] S. Arora, S. Safra, "Probabilistic checking of proofs: A new characterization of NP", *Journal of the ACM* 45 (1): 70-122, 1998.
- [17] S. Arora, C. Lund, R. Motwani, M. Sudan, M. Szegedy, "Proof verification and the hardness of approximation problems", *Journal of the ACM* 45 (3): 501-555, 1998.
- [18] W. Fernandez de la Vega, G.S. Lueker, "Bin packing can be solved within $1 + \epsilon$ in linear time", *Combinatorica (Springer Berlin / Heidelberg)* 1 (4): 349-355, December 1981.
- [19] J. Håstad, "Clique is hard to approximate within $n^{1-\epsilon}$ ", *Acta Mathematica* 182 (1): 105-142, 1999.
- [20] L.A. Levin, (1973). "Universal search problems", *Problems of Information Transmission* 9 (3): 265-266, 1973 (Russian), translated into English by B. A. Trakhtenbrot, "A survey of Russian approaches to perebor (brute-force searches) algorithms", *Annals of the History of Computing* 6 (4): 384-400, 1984.
- [21] T.H. Cormen, C.E. Leiserson, R.L. Rivest, C. Stein, "Introduction to Algorithms", 3. ed., MIT Press 2009.
- [22] L. Fortnow, The status of the P versus NP problem, *Communications of the ACM* 52, no. 9, pp. 78-86, 2009.
- [23] G.J. Woeginger, (15 January 2013) "The P-versus-NP page". <http://www.win.tue.nl/~gwoegi/P-versus-NP.htm>. Retrieved 26 March 2013.